Amir Taylor

Dr.Sarker

COSC 414 Operating Systems

Subsystem B: Amir Taylor Reflection

Building this memory management and virtual memory simulator helped me better understand how operating systems translate logical addresses into physical memory and how page replacement strategies directly affect system performance. One of the biggest design decisions I made was using a shared Sim struct to hold all simulator state, including the frame table, page-to-frame mapping, hit count, and fault count. Since both FIFO and LRU needed access to the same core memory information, centralizing the data avoided duplicated logic and made the simulator easier to extend later.

I also separated address translation into helper functions like pageOf(), offsetOf(), and physical(). This made the code cleaner and prevented repeated bitwise operations throughout the scheduling logic. Using constants for the page size, offset bits, and number of frames also improved readability and made configuration changes much simpler.

For the replacement algorithms, I implemented FIFO using a vector that tracks page load order. This approach models the idea of removing the oldest page currently in memory. LRU required a different strategy, so I used a hash map to record the most recent access time for each page alongside a logical clock counter. Although scanning the map to find the least recently used page is not the most efficient possible implementation, it kept the algorithm understandable and accurate for a simulator of this scale.

One challenge was ensuring that page eviction correctly updated both the frame table and the page mapping table. If one structure changed without the other, stale mappings would remain and produce incorrect address translations. Creating a dedicated load() helper function solved this issue by centralizing all eviction and loading behavior into one place.

Another important design choice was printing the state of physical memory after every access. Seeing frame contents update in real time made it much easier to debug the algorithms and compare FIFO and LRU behavior during the demo sequence. The summary statistics at the end, including hits, faults, and hit percentage, also provided a clear way to evaluate algorithm performance.

The largest limitation of the project is that it models memory management at a simplified level. The simulator assumes fixed-size pages, a single process space, and only four physical frames. It also does not simulate a TLB, dirty pages, disk latency, or true concurrent memory accesses. Given more time, I would expand the project to include additional page replacement strategies such as Optimal or Clock replacement and add support for configurable frame counts and multi-process simulations.